

Inventory audits now reach your books

Slice books-23 — August 20, 2026

Michael, this one is uncomfortable to write, because the headline is a hole that had been open for eleven slices and nobody noticed, including me. I want to start with that rather than bury it, because you asked me to go slow and be meticulous on this particular piece of work and the reason it needed that is the finding itself.

The short version

You had a working inventory-audit posting engine. Seven hundred and sixty-seven lines of it. It knew how to take a finished count, work out what was missing, figure out which inventory account each product category belonged to, and build a balanced journal entry to write the shrinkage off. It was tested. Those tests passed.

Nothing could reach it. Not a button, not a link, not a form, not a background job. The only file in the entire repository that loaded that engine was its own test file. In practice that meant you could open an audit, count the shelf, approve the result, watch the screen say the audit was approved, and your books would never hear about it. The shelf would be corrected. The ledger would sit there holding the old number as though nothing had happened.

That is fixed. There is now a button on the audit page that posts the entry, and it will not appear for anyone but you.

You put your finger on exactly this when you said letting someone process a cycle count with no way to post it to the ledger would have been a killer. It was worse than that — the way to post it existed and had been written months ago. It just was not connected to anything.

What this would have cost you

The number worth holding on to is the one from your Sage 50 exports: \$4,624,697.31 of inventory adjustments entered in a single lazy pass. Inventory shrinkage is the one expense a marijuana retailer can actually deduct, because it lives inside cost of goods sold, and cost of goods sold is the only thing section 280E lets you subtract. Every dollar of real shrink that never reaches the ledger is a dollar of deduction you paid full freight on. An audit that corrects the shelf but not the books does not just leave your balance sheet wrong; it quietly overstates the inventory you are still holding, which overstates your income, which overstates your tax.

So this was not a cosmetic gap. It was the difference between counting your shelf for compliance and counting your shelf for money.

How the work flows now, in your order

You told me exactly how you wanted this to run, and I want to lay it back out so you can check that I built what you described rather than what I found convenient.

You decide what gets counted. The system looks at what it knows — how long since a lot was last counted, how much it is worth, whether anything looks off — and proposes a scope. That proposal shows up on your inventory audit page, which is yours alone. Nobody else sees it. You approve the scope, and approving it is what pushes the count out to the floor.

Your staff count it, and they count blind. The cycle count page no longer has a button to create a count, because creating one is your decision, not theirs. What they get instead is a list of the counts you have already approved and opened. They open one, they count, they enter numbers. They do not see what the system expected to find, they do not see what anything cost, and they do not see a variance. This matters more than it sounds: a counter who can see the expected number is a counter who will find the expected number, and a count that agrees with the books because the counter knew the answer tells you nothing at all.

Any employee can count. That is deliberate and it is what you asked for. It is also the right answer operationally, because the person holding the scanner should not need a manager standing next to them to record a number.

When they finish, it leaves their screen. A completed count disappears from the cycle count page and lands back on your inventory audit page, where there is now a table of finished counts waiting for you. That is the piece that was missing from the original sketch and it is the piece that makes the loop close.

You spot check it, and then you decide. You can open a finished count, see the variances, see what they are worth, record the reason, and send the movement to CCRS the way the system already did. And then, separately, you press Post. Not before. Nothing touches your books until you press that button.

It creates a draft, not a posted entry. You said you wanted the draft created and held for your approval rather than auto-posted, and that is what it does. The screen says so in plain words on the button itself, so there is no guessing about what pressing it will do.

Who can do what

You said anything touching accounting, bookkeeping, taxes or finance should be hard gated to you only, and I took that literally.

Creating an audit, approving an audit, seeing costs and variances, exporting the audit record, and posting to the ledger are all yours. Not your admins, not your managers. Yours. There are six roles in the system and five of them cannot reach those screens; the menu items do not even render for them, so they will not click something and get refused.

Counting is separate, and it is held by the owner, admins, managers and staff. It is specifically **not** held by the reporting-only role. That was a judgement call I want to flag: the obvious shortcut was to let anyone who can open the dashboard record a count, but the reporting-only role exists for someone who should look at numbers and not change them, and recording a count changes the shelf record. So I gave counting its own permission rather than borrowing a broad one.

The compliance calendar is yours now too

You said you did not want employees harassed by the system about payments you have not made or reports you have not filed, and that you would keep that burden yourself. Done. The compliance calendar moved into your Lyman menu, and the overdue banner on the dashboard is now gated to you specifically.

Two details worth knowing. First, the count of overdue items is not merely hidden from other people — it is not even looked up when someone else loads the dashboard. A number that has been fetched and is sitting in memory is one careless edit away from being displayed again, so the query itself is now behind the same gate as the banner. Second, the calendar used to ride on the general settings permission, which your admins hold. They keep that permission and still cannot open the calendar, because the calendar now has a permission of its own. I deliberately did not attach it to the money-accounts permission either; a spot-check on how long you retain camera footage is not a bank account, and labelling it as one would have been convenient today and wrong later.

The database, not just the app

There were two ways into the old cycle-count tables. I closed the one in the application code, which is the one you would have used. The other one was a policy from an old migration that let any logged-in staff member insert, update and delete rows in those tables directly, without going through any of our screens at all. In other words, the retired path was retired in our code and still wide open at the database.

There is a new migration, 0194, that drops those two policies, replaces them with read-only access, and revokes the write permissions outright.

This one needs your hands. Like the last one, it has to be applied manually. After you apply it, there are four review queries at the bottom of the file — query A must come back empty, and query C must come back with something in it. It is safe to run the whole file twice if you are not sure it took the first time; I wrote it so a second run does nothing rather than breaking.

One thing I did not do, and I want to be explicit because the file could have looked stronger if I had lied about it: I did not revoke the service role. That is the key our own server code uses, it ignores database permissions by design, and every remaining reader of those tables depends on it. Revoking it would have broken nine working functions while making the migration look more thorough. The real lock on that path is the refusal we put in the code, and both the migration and its tests say so out loud.

What I got wrong, and how it surfaced

I keep a rule that says a test which went green on its first run is a suspect rather than an achievement, and this slice earned it four separate times. I am telling you about these because you are paying for judgement, and the way I check my own work is the part you cannot see from the outside.

Three of my own tests were passing for no reason whatsoever, and I only found them by putting the entire original codebase back and confirming that my new tests failed against it. Thirty-six of them did. Twenty-two did not, and three of those twenty-two turned out to be worthless.

The most instructive one was a test named for checking that your screen tells you the entry is a draft. It passed against the old

code. The old page did not contain the word "draft" anywhere a human could read — but it did contain the word in a programmer's comment, describing an earlier draft of the file itself. My test read the raw file, found the word in the comment, and reported success. That is precisely the defect this whole slice exists to fix: a description of a thing being mistaken for the thing. I managed to reproduce it inside the test written to prevent it.

A second one asserted that admins cannot open the compliance calendar. True — but it was true before I did anything, because the permission did not exist yet, and asking whether someone holds a permission that was never invented returns no. The test was satisfied by the absence of the very thing it was testing.

A third was named after a specific refusal code and never checked for it, verifying instead something the old code already did in eleven places.

Separately, one of my tests used a piece of regular-expression syntax the project compiler rejects. All fifty-eight tests passed anyway, because the test runner is more forgiving than the compiler. Only running the compiler caught it. A green test suite is not the same thing as a project that builds.

Testing

The full suite is 361 files and 7,212 tests, all passing, none skipped. The compiler is clean, and I proved the compiler was actually running rather than serving a cached result by deliberately breaking a type and watching it complain.

Beyond that, I broke this slice on purpose thirty-four times — unwiring the Post button, handing audit approval back to admins, putting the expected quantity onto the blind count sheet, letting the calendar nag your staff again, leaving the old database policy in place — and confirmed that each sabotage makes a named test fail. All thirty-four were caught. Two of those thirty-four survived on the first attempt and both times the fault was in my test rather than in the code, which is the entire reason for doing it.

What is next

Two things you have already asked for, in your order.

The Sage 50 screens under Reports come out next. They are a window into a system you are leaving, and leaving them in place means every number on your screen has two possible origins.

Then the full walk-through of how the point of sale connects to your books, which you asked for after this: inventory intake and the Cultivera imports landing on the balance sheet, cash sales and cost of goods sold, check payments matching invoices, the time clock feeding payroll, the cards and bank and loans and mortgages, the ATM, and the crypto.

Still waiting on you

Nothing here is blocked on these, but the list does not get shorter on its own: the exact tax year of your S-election, the ending accumulated adjustments account figure from your Schedule M-2, the per-shareholder amount for the section 6699 penalty, and the quarterly federal short-term rates. Beyond that, the depreciation schedule and asset list, your grandfather's work papers, the twelve years of returns, and the intercompany detail.

And the two migrations that need applying by hand: 0193, the one that makes the security log yours alone, and 0194 from this slice. For 0193 the middle review query must come back empty. For 0194 it is query A that must be empty and query C that must not be.